

Website CEIS Caramulo

Documentação técnica organizada para manutenção, operação e continuidade do website institucional e do backoffice.

PROJETO

CEIS Caramulo

STACK PRINCIPAL

Next.js, React, Prisma, PostgreSQL, Auth0, Vercel

DOMÍNIOS DE PRODUÇÃO

ceiscaramulo.pt / www.ceiscaramulo.pt

ÚLTIMA VALIDAÇÃO TÉCNICA

02/07/2026

COMMIT VALIDADO

4fbf669 - Remove local CMS file fallback

ESTADO DO DEPLOY

Vercel Production READY

Este documento não inclui segredos, passwords, tokens ou valores reais de variáveis de ambiente. Esses dados devem permanecer apenas na Vercel/Auth0/Firebase/gestor seguro da empresa.

Índice

1. Resumo executivo
2. Arquitetura geral
3. Tecnologias e dependências
4. Estrutura do projeto
5. Backoffice
6. Conteúdos públicos
7. Base de dados
8. Autenticação e permissões
9. Ficheiros, imagens e galerias
10. Variáveis de ambiente
11. Deploy e domínios
12. Comandos de desenvolvimento
13. Testes e validação
14. Manutenção recomendada
15. Resolução de problemas
16. Checklist de entrega

1. Resumo executivo

O website CEIS Caramulo é uma aplicação web institucional com páginas públicas, sistema de conteúdos e backoffice administrativo. A aplicação permite gerir notícias, atividades, recursos/publicações, mensagens recebidas, aparência do site, página Sobre Nós, utilizadores administrativos e galerias associadas a páginas específicas.

A solução foi preparada para produção em Vercel, com base de dados PostgreSQL gerida por Prisma, autenticação Auth0 e armazenamento de ficheiros através de Vercel Blob e rotas internas de assets. O foco técnico atual é manter o site leve, evitar transferências excessivas de origem e impedir que ficheiros grandes sejam guardados diretamente na base de dados.

Estado no momento da entrega: testes automatizados, TypeScript, build de produção e verificação HTTP local foram executados com sucesso. O deploy de produção na Vercel ficou no estado **READY**.

2. Arquitetura geral

CAMADA	RESPONSABILIDADE	LOCALIZAÇÃO PRINCIPAL
Frontend público	Páginas institucionais, notícias, atividades, recursos, galerias e contactos.	app/ , components/
Backoffice	Gestão administrativa de conteúdos, aparência, mensagens, utilizadores e galerias.	app/backoffice/page.tsx
API interna	Rotas para CRUD de conteúdos, sessão admin, layout, uploads, galeria, newsletter e mensagens.	app/api/
Dados	Persistência em PostgreSQL via Prisma, com modelos para conteúdos, utilizadores, layout e mensagens.	prisma/schema.prisma , app/api/_lib/cms.ts
Uploads	Armazenamento otimizado de imagens, documentos e media, com Vercel Blob e rotas locais de leitura.	lib/upload-storage.ts , app/uploads/backoffice/ , app/api/gallery/

3. Tecnologias e dependências

TECNOLOGIA	USO NO PROJETO
Next.js 15	Aplicação principal com App Router, páginas públicas, rotas API e build de produção.
React 18	Componentes de interface e backoffice.
TypeScript	Tipagem estática do projeto.
Prisma 7	Acesso à base de dados PostgreSQL e geração de cliente Prisma.
PostgreSQL	Base de dados principal em produção.
Auth0	Autenticação administrativa do backoffice.
Firebase Admin	Suporte a credenciais/admin em fluxos existentes.
Vercel Blob	Armazenamento de ficheiros/media para reduzir peso na base de dados e evitar limites de origem.
Vitest	Testes automatizados de páginas, APIs, helpers e regressões.
Tailwind CSS / Radix UI / Lucide	Interface, componentes, estilos e ícones.

4. Estrutura do projeto

PASTA/FICHEIRO	DESCRIÇÃO
app/	Rotas públicas, backoffice e API routes do Next.js.
components/	Componentes reutilizáveis do frontend e backoffice.
lib/	Funções utilitárias, autenticação, storage, layout e lógica partilhada.
data/	Conteúdos estáticos/fallbacks usados quando necessário.
prisma/	Schema e migrações da base de dados.
src/generated/prisma/	Cliente Prisma gerado automaticamente.
tests/	Testes automatizados do projeto.
public/	Assets estáticos servidos diretamente pelo Next.js.
.env / .env.local	Configuração local. Os valores reais não devem ser partilhados publicamente.

5. Backoffice

O backoffice está disponível em `/backoffice` e a página de entrada em `/backoffice/login`. O acesso é protegido por autenticação e permissões. Sem sessão válida, as rotas administrativas respondem com `401 Não autorizado`, que é o comportamento esperado.

5.1 Áreas disponíveis

ÁREA	FINALIDADE
Visão geral	Resumo do estado do backoffice, contagens e acessos às áreas principais.
Perfil	Dados do utilizador autenticado e alteração de palavra-passe.
Notícias	Criação, edição, publicação e imagem de capa de notícias.
Atividades	Gestão de atividades/eventos, datas, categorias, locais e imagem.
Recursos	Biblioteca/recursos/publicações, documentos PDF, capas e metadados.
Mensagens	Consulta e marcação de mensagens enviadas pelo formulário de contacto.
Galerias por página	Media associados a páginas como Oficina do Burel, Artigos para venda, Biblioteca JRS, PON do Jueus, Escola dos Nossos Avós, Oficinas de Formação, Publicações e Recursos.
Administradores	Gestão de utilizadores administrativos, estado ativo e permissões.
Histórico	Registo de alterações administrativas e retenção dos eventos recentes.
Sobre Nós	Edição do conteúdo da página Sobre Nós e dados visíveis na barra de contacto.
Aparência	Configuração de textos, páginas institucionais, rodapé, contactos e blocos editáveis do site.

5.2 Experiência de utilização

- Os formulários usam mensagens de erro/sucesso para orientar o utilizador.
- Uploads em lote mostram progresso, reduzindo a sensação de bloqueio em ficheiros maiores.
- As listagens administrativas são limitadas para evitar carregar demasiados registos de uma vez.
- O backoffice carrega secções conforme necessário, em vez de puxar todos os dados logo na entrada.
- Áreas sensíveis só aparecem quando o administrador tem permissões adequadas.

6. Conteúdos públicos

O site público é composto por páginas institucionais, conteúdos editoriais e páginas alimentadas pelo backoffice. As principais rotas públicas incluem:

- `/` - página inicial.
- `/sobre-nos` - apresentação institucional.
- `/contactos` - contactos e formulário.
- `/noticias` e `/noticias/[slug]` - notícias.
- `/atividades` e `/atividades/[id]` - atividades.
- `/biblioteca` e `/biblioteca/[id]` - recursos/biblioteca.
- `/oficina-do-burel`, `/artigos-para-venda`, `/biblioteca-jrs`, `/pon-do-jueus`, `/escola-dos-nossos-avos`, `/oficinas-de-formacao` e `/publicacoes` - páginas programáticas com galerias/conteúdos associados.

O conteúdo público deve continuar a ser editado preferencialmente através das áreas próprias do backoffice. Alterações manuais diretamente no código só devem ser feitas quando se tratar de estrutura, comportamento ou design não editável.

7. Base de dados

A base de dados usa PostgreSQL com Prisma. O schema principal encontra-se em `prisma/schema.prisma`. Os modelos mais relevantes são:

MODELO	DESCRIÇÃO
<code>News</code>	Notícias, slugs, conteúdo rico, categoria, autor e estado publicado.
<code>Activity</code>	Atividades com datas, local, categoria e imagem.
<code>Publication</code>	Recursos/publicações, documentos PDF e capas.
<code>ContactMessage</code>	Mensagens do formulário de contacto.
<code>SiteSetting</code>	Chave/valor para dados de compatibilidade e armazenamento auxiliar.
<code>SiteLayout</code>	Configurações globais de aparência/layout do site.
<code>AdminUser</code>	Utilizadores administrativos e respetivo estado.
<code>AdminAuditLog</code>	Histórico de ações administrativas.
<code>NewsletterSubscriber</code>	Subscritores da newsletter.
<code>GalleryMedia</code>	Media de galeria global antigo. As galerias por página usam também armazenamento otimizado em settings/blob.

7.1 Cuidados com a base de dados

- Não guardar ficheiros grandes diretamente na base de dados.
- Manter documentos/imagens/media em Vercel Blob ou rotas de upload já existentes.
- Executar migrações apenas quando houver alterações reais ao schema.
- Validar sempre com `pnpm test:run` e `pnpm build` antes de produção.

8. Autenticação e permissões

A autenticação administrativa usa Auth0. O backoffice depende de sessão válida e valida permissões antes de permitir operações sensíveis.

ELEMENTO	DESCRIÇÃO
<code>/auth/[auth0]</code>	Rotas de autenticação Auth0.
<code>/api/admin/session</code>	Valida a sessão administrativa atual.
<code>/api/admin/users</code>	Gestão de utilizadores administrativos.
<code>/api/admin/password</code>	Alteração de palavra-passe do utilizador autenticado.

8.1 Permissões administrativas

As permissões principais usadas pelo backoffice incluem:

- `news` - notícias.
- `activities` - atividades.
- `publications` - recursos/publicações.
- `contacts` - mensagens de contacto.
- `gallery` - galerias/media.
- `layout` - aparência/layout e páginas institucionais.
- `admins` - gestão de administradores.
- `audit` - histórico de alterações.

Ao alterar domínios em produção, confirmar sempre no Auth0 os campos **Allowed Callback URLs**, **Allowed Logout URLs** e **Allowed Web Origins**. Incompatibilidades nestes campos podem causar erros de autenticação ou estado inválido.

9. Ficheiros, imagens e galerias

O projeto foi otimizado para evitar saturar a base de dados e para reduzir problemas de transferência de origem na Vercel. O princípio geral é: a base de dados guarda metadados e URLs; os ficheiros ficam em storage apropriado.

9.1 Uploads de conteúdos

- Imagens de notícias, atividades, publicações e layout usam `storeUploadedFile`.
- O armazenamento preferencial é Vercel Blob via `storePublicUpload`.
- Rotas como `/uploads/backoffice/[...path]` servem ficheiros antigos/compatibilidade.
- Imagens HEIC/HEIF são bloqueadas para evitar incompatibilidade nos browsers.

9.2 Galerias do backoffice

As galerias por página suportam ficheiros conforme o contexto. O fluxo recente foi otimizado para detetar automaticamente o tipo do ficheiro e processar uploads de forma estável.

CONTEXTO	USO	TIPOS ESPERADOS
<code>oficina-do-burel</code>	Media da Oficina do Burel.	Fotos, vídeos, áudios e documentos.
<code>artigos-para-venda</code>	Media dos Artigos para venda.	Fotos e documentos. Vídeo/áudio foram removidos desta área.
<code>biblioteca-jrs</code>	Media da Biblioteca José Ribeiro dos Santos.	Fotos, vídeos, áudios e documentos.
<code>pon-do-jueus</code>	Media da página PON do Jueus.	Fotos, vídeos, áudios e documentos.
<code>escola-dos-nossos-avos</code>	Media da Escola dos Nossos Avós.	Fotos, vídeos, áudios e documentos.
<code>oficinas-de-formacao</code>	Media das Oficinas de Formação.	Fotos, vídeos, áudios e documentos.
<code>publicacoes</code>	Media da página Publicações.	Fotos, vídeos, áudios e documentos.
<code>biblioteca</code>	Recursos/biblioteca.	Fotos, vídeos, áudios e documentos.

9.3 Boas práticas para ficheiros

- Preferir imagens em `.jpg`, `.png` ou `.webp`.
- Comprimir imagens grandes antes de carregar.
- Usar PDFs otimizados para documentos.
- Evitar carregar vídeos muito pesados quando não forem essenciais.
- Verificar se os ficheiros aparecem no frontend depois de guardar no backoffice.

10. Variáveis de ambiente

As variáveis existem em ambiente local e na Vercel. Os valores reais devem ser tratados como confidenciais.

VARIÁVEL	FUNÇÃO
<code>DATABASE_URL</code> , <code>POSTGRES_URL</code> , <code>PRISMA_DATABASE_URL</code>	Ligação à base de dados PostgreSQL/Prisma.
<code>APP_BASE_URL</code> , <code>NEXT_PUBLIC_SITE_URL</code>	URL base usada pela aplicação e serviços externos.
<code>AUTH0_DOMAIN</code> , <code>AUTH0_CLIENT_ID</code> , <code>AUTH0_CLIENT_SECRET</code> , <code>AUTH0_SECRET</code>	Configuração Auth0.
<code>AUTH0_MANAGEMENT_CONNECTION</code>	Ligação usada para gestão de utilizadores Auth0.
<code>FIREBASE_AUTH_API_KEY</code> , <code>FIREBASE_PROJECT_ID</code> , <code>FIREBASE_CLIENT_EMAIL</code> , <code>FIREBASE_PRIVATE_KEY</code>	Credenciais Firebase/Admin.
<code>BLOB_STORE_ID</code> , <code>BLOB_READ_WRITE_TOKEN</code> , <code>VERCEL_OIDC_TOKEN</code>	Armazenamento Vercel Blob e acesso seguro a storage.
<code>RESEND_API_KEY</code> , <code>MAIL_FROM</code> , <code>RESEND_MAIL_FROM</code>	Envio de emails/newsletter via Resend.
<code>ALLOW_ADMIN_SELF_REGISTRATION</code>	Controlo de auto-registo administrativo, se ativo.

Nunca colocar tokens ou chaves reais em documentação, emails ou mensagens. Em caso de exposição accidental, revogar e gerar novas credenciais.

11. Deploy e domínios

O deploy de produção é feito através da Vercel a partir do branch `main` do GitHub.

ITEM	VALOR
Projeto Vercel	<code>ceiscaramulo</code>
Branch de produção	<code>main</code>
Domínio principal	<code>ceiscaramulo.pt</code>
Domínio com www	<code>www.ceiscaramulo.pt</code>
Domínio Vercel	<code>ceiscaramulo-ten.vercel.app</code>
Último commit validado	<code>4fbf66923023ac51c23f9eb778dce870823b83a4</code>

11.1 Fluxo recomendado de publicação

1. Fazer alterações no código.
2. Executar testes e build localmente.
3. Fazer commit com mensagem clara.
4. Fazer push para `main`.
5. Aguardar o deployment automático na Vercel.
6. Confirmar estado `READY`.
7. Validar as páginas principais no browser.

12. Comandos de desenvolvimento

COMANDO	DESCRIÇÃO
<code>pnpm install</code>	Instala dependências e gera Prisma Client via postinstall.
<code>pnpm dev</code>	Arranca o servidor de desenvolvimento.
<code>pnpm build</code>	Cria build de produção e valida tipos/lint do Next.
<code>pnpm start</code>	Arranca a aplicação compilada em modo produção.
<code>pnpm test:run</code>	Executa todos os testes automatizados.
<code>pnpm exec tsc --noEmit --pretty false</code>	Valida TypeScript sem gerar ficheiros.
<code>pnpm prisma:generate</code>	Gera o Prisma Client.
<code>pnpm prisma:migrate</code>	Cria/aplica migrações em desenvolvimento.
<code>pnpm prisma:push</code>	Sincroniza schema com base de dados quando aplicável.

12.1 Arranque local recomendado

```
pnpm install
pnpm prisma:generate
pnpm dev
```

12.2 Verificação antes de entrega

```
pnpm test:run
pnpm exec tsc --noEmit --pretty false
pnpm build
```

13. Testes e validação

A suíte de testes cobre rotas API, backoffice, autenticação, uploads, rich text, páginas públicas, layout, galeria, newsletter e regressões de performance/origin transfer.

VALIDAÇÃO EXECUTADA	RESULTADO
<code>pnpm test:run</code>	71 ficheiros de teste, 328 testes passaram.
<code>pnpm exec tsc --noEmit --pretty false</code>	Sem erros TypeScript.
<code>pnpm build</code>	Build de produção concluído com sucesso.
Verificação local HTTP	<code>/backoffice/login</code> , <code>/backoffice</code> , <code>/api/health</code> e <code>/api/layout</code> responderam 200.
Rotas admin sem sessão	Responderam 401, comportamento esperado.
Vercel production	Deployment READY.

14. Manutenção recomendada

14.1 Rotina semanal

- Entrar no backoffice e confirmar que as áreas principais carregam.
- Verificar se as galerias e imagens públicas renderizam corretamente.
- Consultar mensagens de contacto novas.
- Confirmar se não há deploys falhados na Vercel.
- Verificar uso de Vercel Blob, base de dados e origin transfer.

14.2 Rotina antes de alterações importantes

- Fazer backup/export da base de dados, se a alteração envolver schema ou dados em massa.
- Confirmar que as variáveis da Vercel estão completas.
- Executar testes e build local.
- Guardar screenshots das páginas críticas antes/depois, quando a alteração for visual.

14.3 Boas práticas de conteúdo

- Usar títulos claros e curtos.
- Evitar imagens demasiado pesadas.
- Preencher descrições e textos alternativos sempre que possível.
- Publicar apenas conteúdo revisto.
- Testar o frontend depois de guardar alterações no backoffice.

15. Resolução de problemas

SINTOMA	CAUSA PROVÁVEL	AÇÃO RECOMENDADA
Backoffice não abre ou volta ao login.	Configuração Auth0/domínio/callback incorreta ou sessão expirada.	Confirmar <code>AUTH0_*</code> , domínio atual, callbacks no Auth0 e cookies/sessão.
Erro "state parameter is invalid".	URL/callback Auth0 desalinhado, cookies antigos ou domínio errado.	Limpar cookies, confirmar <code>APP_BASE_URL</code> e callbacks Auth0 para o domínio usado.
Upload em lote demora muito.	Ficheiros grandes ou ligação lenta.	Comprimir ficheiros e usar o progresso de upload para acompanhar. Confirmar Vercel Blob ativo.
Imagem aparece como quebrada no frontend.	Asset antigo, URL inválida ou storage indisponível.	Testar a URL direta, verificar <code>/uploads/backoffice/...</code> ou <code>/api/gallery/assets/...</code> e voltar a carregar a imagem se necessário.
Vercel mostra uso elevado de transferência de origem.	Imagens/media pesados ou assets dinâmicos servidos repetidamente.	Otimizar imagens, evitar vídeos pesados e manter Blob/storage em vez de base de dados.
Base de dados lenta ou indisponível.	Limite/quota, conexão ou manutenção do fornecedor.	Verificar painel da base de dados, logs da Vercel e variáveis <code>DATABASE_URL</code> / <code>PRISMA_DATABASE_URL</code> .
Newsletter não envia emails.	Chave Resend ou remetente/domínio não verificado.	Confirmar <code>RESEND_API_KEY</code> , <code>MAIL_FROM</code> e domínio verificado no Resend.
Deploy falha na Vercel.	Erro de build, env em falta ou problema de dependência.	Ver logs do deployment, correr <code>pnpm build</code> localmente e comparar variáveis de ambiente.

16. Checklist de entrega

ITEM	ESTADO
Projeto em GitHub com branch <code>main</code> atualizado.	Concluído
Deploy de produção Vercel no estado READY.	Concluído
Domínios principais associados ao deployment.	Concluído
Backoffice com autenticação e permissões.	Concluído
Galerias e uploads otimizados para não pesar a base de dados.	Concluído
Testes automatizados executados.	Concluído
Build de produção executado.	Concluído
Documentação técnica em PDF.	Concluído

17. Recomendações finais

- Manter sempre os uploads de media fora da base de dados sempre que possível.
- Antes de instalar novas bibliotecas, avaliar impacto no bundle e no build.
- Não alterar variáveis Auth0/Vercel sem validar login e callbacks.
- Em alterações ao backoffice, atualizar testes no mesmo commit.
- Para alterações visuais importantes, validar desktop e mobile.
- Guardar acessos administrativos em gestor de passwords da empresa.